

Un catalogue, un bibliothécaire, une réponse à expliquer.

Raconte le même parcours pendant 15 minutes : préparer le catalogue, retrouver les passages, puis rédiger et contrôler la réponse. Chaque fiche traduit l'image en termes techniques et te dit quoi montrer.

État vérifié aujourd'hui

76 tests passent ; Docker et Swagger fonctionnent. La démo directe a été rétablie avec minstral-8b-2512. Les scores de juillet concernent l'ancien modèle. La démo de secours est explicitement enregistrée. Le corpus couvre juillet 2025 → juillet 2026 ; il ne représente pas un agenda actualisé pour aujourd'hui.

Un catalogue culturel, une réponse vérifiable

L'idée à retenir : Le visiteur pose une question ; nous cherchons dans des fiches réelles avant de répondre.

Bonjour Jérémy. Pour expliquer ce POC, je vous propose une image simple : une bibliothèque d'événements culturels. Un visiteur arrive avec une demande, par exemple une sortie en famille à Paris. Il souhaite une réponse utile et des informations qu'il peut vérifier. Mon travail a été de constituer le catalogue, de préparer ses fiches pour la recherche, puis de relier cette recherche à un modèle qui rédige et à une API que vos équipes peuvent appeler. Le catalogue livré contient 7 586 événements parisiens. Je vais suivre ce parcours avec vous, montrer deux demandes en direct et expliquer ce qui fonctionne ainsi que ce qui reste à améliorer.

À montrer / à faire

Partage uniquement la fenêtre du diaporama. Garde ce guide hors du partage.

Transition à dire : « Pour comprendre la solution, séparons le catalogue, la recherche et la rédaction. »

Chercher dans le catalogue avant de répondre

L'idée à retenir : Le RAG apporte des passages au rédacteur au moment de la question.

Dans notre bibliothèque, les fiches OpenAgenda forment le catalogue. Chaque fiche décrit un événement, avec son titre, son lieu, ses dates et son lien. Le bibliothécaire représente la recherche avec FAISS : il retrouve des passages proches de la demande. Puis un rédacteur, le modèle Mistral, reçoit ces passages et formule la réponse. Les embeddings sont ce qui permet de comparer numériquement le sens des textes et de la question. Le RAG consiste à chercher d'abord, puis à rédiger avec les informations retrouvées. Je n'ai pas entraîné Mistral sur le catalogue : j'ai construit l'index et organisé ce passage d'informations. L'analogie a une limite : FAISS compare des vecteurs, il ne comprend pas toutes les contraintes comme un bibliothécaire humain.

À montrer / à faire

Pointe les trois étapes. L'exemple famille existe dans le jeu RAGAS ; ce n'est pas une nouvelle preuve live.

Transition à dire : « Avant d'aider un visiteur, il faut constituer un catalogue fiable. »

Constituer et nettoyer le catalogue

L'idée à retenir : Conserver des fiches lisibles et leurs références pour vérifier les réponses.

J'ai commencé par collecter les fiches du dataset public OpenAgenda indiqué par la mission, accessible via Opendatasoft. J'ai choisi Paris et une fenêtre du 10 juillet 2025 au 10 juillet 2026. La collecte parcourt les pages, retire les doublons par identifiant et nettoie les descriptions, notamment les balises HTML. Je garde les informations qui permettent de vérifier une réponse : titre, description, lieu, dates et URL. Les données sont exportées dans un fichier Parquet, un format structuré relisible par les scripts. Ce catalogue contient 7 586 événements. Comme une bibliothèque dont le catalogue n'a pas été mis à jour depuis juillet, il manque les nouvelles fiches d'octobre. Je reconnais cette limite : pour conseiller des sorties actuelles, il faut actualiser les données et vérifier les dates.

À montrer / à faire

Explique la fenêtre telle qu'elle est. Si on te demande le fichier, ouvre le manifeste de collecte, pas le JSON brut entier.

Transition à dire : « Une fiche peut être longue : nous allons préparer des passages plus faciles à retrouver. »

03:05 → 04:20 • 75 secondes • repère incluant gestes et pauses

Découper les fiches, puis comparer leur sens

L'idée à retenir : Une fiche longue devient plusieurs passages, chacun relié à son événement.

Dans un catalogue, certaines fiches sont courtes et d'autres très longues. Pour ne pas transmettre toute une longue description au rédacteur, je les découpe en passages, appelés chunks. Le réglage de départ est de 1 000 caractères, avec jusqu'à 200 caractères de chevauchement pour préserver les informations aux frontières. Le découpage cherche à respecter les séparateurs du texte. La fiche CHIMERE illustre ce point : elle produit 15 passages. Tout le catalogue produit 14 903 passages, et non 14 903 événements. Chaque passage est transformé par mistral-embed en un vecteur de 1 024 nombres. Ces nombres servent à comparer le sens ; ce ne sont pas des catégories lisibles par un humain. Je conserve aussi le lien vers la fiche, ses dates et son lieu.

À montrer / à faire

Distingue la fiche d'exemple et les volumes de tout le corpus. Aucun réindexage en direct.

Transition à dire : « Voyons comment le bibliothécaire utilise ces représentations pour une question. »

Le bibliothécaire cherche ; le rédacteur répond

L'idée à retenir : Quatre passages retrouvés deviennent le contexte du rédacteur.

Quand le visiteur pose une question, mistral-embed la transforme avec le même modèle que les passages. Les vecteurs deviennent ainsi comparables. FAISS retrouve les quatre passages les plus proches : c'est le top-k fixé à quatre. Dans notre image, le bibliothécaire dépose quatre extraits sur le bureau du rédacteur. Le prompt est sa consigne : répondre en français à partir de ces extraits, citer les informations disponibles et dire qu'il ne sait pas si elles manquent. Mistral Small était le modèle initial. Aujourd'hui, il renvoyait une limite d'usage ; j'ai configuré la version ministral-8b-2512, puis vérifié la démo. L'embedding reste identique, donc le catalogue vectoriel n'a pas été reconstruit. Les scores historiques concernent toutefois Small. Le prompt aide le rédacteur, mais ne garantit pas une réponse sans erreur.

À montrer / à faire

Pointe la séparation embedding/chat et le texte du prompt. Si demandé, ouvre SYSTEM_PROMPT dans rag.py.

Transition à dire : « Cette bibliothèque fonctionne en deux temps : la préparation et le traitement d'une demande. »

Préparer les rayons une fois, chercher à chaque demande

L'idée à retenir : La préparation du catalogue et la réponse à une question sont deux étapes distinctes.

Il faut distinguer les deux moments. Avant les questions, je prépare les rayons : collecte, nettoyage, découpage, calcul des embeddings et enregistrement de l'index FAISS. Chaque passage reste relié à sa fiche. Puis, à chaque demande, je vectorise uniquement la question, recherche les passages et appelle le rédacteur. Je ne recollecte pas tout OpenAgenda et je ne recalculerai pas tous les vecteurs à chaque question. Le fichier FAISS livré contient 14 903 vecteurs ; tous les identifiants d'événements sont représentés. L'index est chargé à la première demande, puis réutilisé. Attention, les rayons sont locaux, mais mistral-embed et le rédacteur sont des services distants. Cette bibliothèque dépend donc encore d'internet.

À montrer / à faire

Si Jérémy demande le code, montre service.py puis rag.py. Évite de naviguer entre toutes les fonctions.

Transition à dire : « Pour que vos équipes puissent poser une question, il reste à ouvrir un guichet. »

Un guichet pour les équipes produit

L'idée à retenir : L'API ouvre un accès au catalogue, à la recherche et à la rédaction.

FastAPI est le guichet de notre bibliothèque. Une application lui envoie une question en JSON et reçoit une réponse, des sources et le nombre de passages récupérés. La route ask fait ce travail. Health indique que le guichet répond et affiche sa configuration, mais ne vérifie pas à elle seule que le bibliothécaire et le rédacteur peuvent répondre. Rebuild reconstruit les rayons à partir du Parquet ; elle ne collecte pas de nouvelles fiches. Swagger documente ces routes. Docker rassemble le code, les dépendances et le catalogue de départ pour lancer le même service ailleurs. La clé Mistral reste côté serveur. Le jeton de reconstruction est optionnel dans le POC ; il faut sécuriser ce guichet avant de l'exposer publiquement.

À montrer / à faire

Pendant la démo, ouvre Swagger et montre les routes ; garde /rebuild fermé. Tu peux cliquer le lien Démo de la slide suivante.

Transition à dire : « Passons maintenant au guichet avec deux demandes concrètes. »

Au guichet : une fiche trouvée, une demande hors catalogue

L'idée à retenir : Observer le trajet d'une vraie demande, puis contrôler la réponse avec la fiche.

Je montre d'abord les routes et la configuration du guichet. Dans la page de démonstration, je demande des informations sur Concert Fishers à Paris et je clique sur l'appel direct. La page transmet la question à l'API, qui recherche les passages et appelle le rédacteur. Je compare maintenant la réponse à la fiche : Concert Fishers, Le Gymnase Montparnasse, le 21 juin 2026 et l'URL OpenAgenda. Cet événement est passé ; il sert à vérifier la chaîne sur le catalogue livré. Pour la deuxième demande, je cherche des expositions photo à Lyon. Notre catalogue couvre Paris. Le modèle indique qu'il ne sait pas répondre à partir de ce corpus. Le bibliothécaire retrouve tout de même des passages voisins : ce ne sont pas des recommandations pour Lyon. Cette distinction explique pourquoi un filtre de ville serait utile.

À montrer / à faire

140 s au total : 20 s health/Swagger ; 50 s cas nominal ; 40 s Lyon ; 30 s explication et retour au deck. Une tentative live maximum en cas de quota, puis enregistrement. Mode secours disponible hors réseau.

Transition à dire : « Deux demandes donnent des exemples. Comment vérifier plus largement la bibliothèque ? »

Tester le mécanisme et examiner les réponses

L'idée à retenir : Les tests du guichet et des rayons ne suffisent pas à juger toutes les réponses.

Pour contrôler la bibliothèque, je distingue la mécanique et la qualité des conseils. Les 76 tests locaux passent : ils vérifient notamment les filtres, le nettoyage, le découpage, l'indexation, le contrat API et les erreurs. Ils utilisent des doublures pour ne pas dépendre du fournisseur à chaque exécution. Les appels réels sont vérifiés séparément dans la démo. Le jeu fonctionnel contient quatre questions annotées, avec des titres attendus et un cas de refus. Sa règle est permissive : un titre présent dans les sources peut suffire à classer le cas comme correct, même si la réponse reste imparfaite. Quatre cas corrects ne signifient donc pas cent pour cent de fiabilité. Aucune chaîne CI/CD n'est livrée ; les scripts permettent déjà d'automatiser les vérifications.

À montrer / à faire

Annonce le résultat des tests, puis explique ce qu'ils couvrent. Montre le petit jeu CSV si demandé.

Transition à dire : « RAGAS apporte une seconde lecture : examiner les extraits et le travail du rédacteur. »

Le lecteur contrôle les extraits et la rédaction

L'idée à retenir : Quatre métriques apportent quatre regards différents sur trois réponses.

J'ai aussi utilisé RAGAS sur trois cas, avec les passages effectivement récupérés, la réponse et une référence humaine. Reprenons la bibliothèque. La fidélité demande si les affirmations du rédacteur sont soutenues par les extraits : sa moyenne est 0,900. La pertinence demande si la réponse traite la demande du visiteur : 0,846. La précision du contexte examine l'utilité et l'ordre des extraits apportés par le bibliothécaire : 1,000. Ce score ne signifie pas qu'il a retrouvé tous les événements pertinents. La correction compare la réponse à la référence attendue : 0,683. Une réponse peut donc être cohérente avec les extraits, tout en oubliant une information ou en s'éloignant de la référence. Le cas famille est le plus faible en correction, autour de 0,581. Ces résultats datent du 23 juillet, utilisent Mistral Small et ne portent que sur trois cas. Ils orientent les améliorations ; ils ne démontrent pas la qualité générale de Ministral 8B.

À montrer / à faire

Lis les quatre scores avec leur sens, puis donne la limite du petit échantillon. En discussion, ouvre les exemples RAGAS du rapport.

Transition à dire : « Ces contrôles mettent en évidence trois limites concrètes de notre bibliothèque. »

Un catalogue daté, une recherche imparfaite, un rédacteur externe

L'idée à retenir : L'analogie permet aussi de comprendre où le système peut échouer.

La première limite concerne le catalogue : il est limité à Paris et figé en juillet. Il faut actualiser les fiches et filtrer les occurrences pour recommander des sorties actuelles. La deuxième concerne le bibliothécaire : il apporte les passages les plus proches, même lorsque leur ville ou leur date ne répond pas à la demande. Un seuil calibré et des filtres explicites rendraient le refus plus robuste. La troisième concerne le rédacteur : il dépend de Mistral, peut rencontrer une limite d'usage et peut encore ajouter un conseil hors contexte. J'ai rétabli la démo avec Ministral 8B et contrôlé les erreurs fournisseur, mais cela ne prouve pas une qualité équivalente à Small. Il faut réévaluer ce modèle, diversifier les questions et renforcer les contrôles avant un déploiement plus large.

À montrer / à faire

Relie chaque limite à une amélioration. Si la démo enregistrée a été utilisée, rappelle simplement la cause observée.

Transition à dire : « La prochaine étape est donc d'améliorer cette bibliothèque et de mesurer son utilité avec vos équipes. »

Une bibliothèque testable ; un pilote à mesurer

L'idée à retenir : Le POC livre le parcours complet ; le service doit encore être amélioré et évalué.

Pour conclure, le POC livre notre bibliothèque : un catalogue réel, des passages indexés, une recherche par proximité de sens, un rédacteur guidé par les sources et un guichet API accessible avec Docker. Je propose d'abord d'actualiser le catalogue et d'ajouter les contraintes de ville et de date. Ensuite, d'élargir les références humaines et de comparer les réglages ainsi que les modèles. Enfin, un pilote doit mesurer l'utilité des réponses, les refus corrects, les clics vers les sources, la latence et le coût. Je vous montre brièvement le rapport et le dépôt : scripts permet de relancer les étapes, src contient le moteur et le guichet, tests les contrôles, et seed-data le catalogue de référence et son index. La solution est examinable et testable ; sa qualité à plus grande échelle reste à mesurer. Je suis prêt à répondre à vos questions.

À montrer / à faire

30 s de conclusion ; 20 s rapport (architecture, évaluation, limites) ; 25 s dépôt (scripts/, src/, tests/, seed-data/).
Retourne ensuite au deck. Tous les liens sont dans le guide.

Transition à dire : « Discussion : commence par la réponse courte, puis explique le composant et la preuve concernés. »

Guide et diaporama synchronisés sur les mêmes 12 étapes. Les supports précédents sont conservés dans archives/.